# CEPARCO Project Proposal

Group 6 Members:

1. Bactong, Gabrielle Joei V.
2. Lobo, Shirley Marie A.
3. Panganiban, Diane
4. Gunita, Catherine Rosswyn D.

## 01. Abstract

The SVM algorithm is widely used in machine learning training. It is utilised in classification tasks and regression analysis. The aim of this project will be to adopt the study by Suarez et al. (2025), which focuses on optimising SVM using a linear regression approach. The same dataset used by Suarez et al. (2025) will be utilised in this project. The group will use YMM SMID registers and CUDA to implement the equations used in the linear regression approach. This addresses the bottlenecks identified. In this study, the linear optimisation problem 7 feasibility check will not be parallelised and will essentially remain sequential. The expected output of the project will be the reduced and filtered training dataset, performance and execution time speedup matrices of the parallel approach, as well as the sanity check of the training dataset filtered through SIMD/SIMT. A C implementation will also be implemented for the purpose of comparison with the resulting matrices of the parallel approach.

## 02. Description of Proposed Project

Support Vector Machines (SVMs) are widely used in machine learning training. They are used for classification tasks and regression analysis that can help with image classification, fraud detection, text classification, recommender systems, etc (Mun, 2023). However, the time it takes to train SVMs scales significantly with the number of data points $N$. Every data point is a decision variable, causing SVMs have a time complexity of $O(N^3)$ (Abdiansah & Wardoyo, 2015; Suárez et al., 2025). A study by Suárez et al. (2025) approached this problem by removing points through linear programming that have no chance to be support vectors, reducing $N$ and speeding up training. Thus, our group aims to build off their research and to parallelize the remaining stages in their SVM pipeline, combining linear programming and SIMD/SIMT programming to optimize training.

### a. Inputs to the project

The main input to be used for the project is the same training dataset used by Suárez et al (2025). The dataset is an $N \times D$ matrix with columns $x1$, $x2$, $x3$, $x4$, and labels $y$. The dataset is accessible through this GitHub link: Dataset.

### b. Proposed Process

#### i. Processes to be parallelized

The processes to be parallelized using AVX2 are the following: (1) Barycenters (Geometric) Calculation, and (2) filtering of the barycenters. Meanwhile, the point-in-convex hull tests will be parallelized using SIMT (CUDA) programming.

#### ii. Existing Implementations

Over the years, attempts to optimize SVM algorithms have been proposed and studied by scholars. The identified implementations are honourable mentions from different approaches. Among the published studies, the linear regression method for optimising SVM, such as the method proposed by Suarez et al. (2025), has become the focus of our project. It essentially integrates linear programming techniques with SVM to classify optimal separators between sets of points. Another implementation is an optimisation-focused approach by Woodsend, K., & Gondzio, J. (2011), in which they introduce an interior point method for linear SVM training which takes advantage of the Hessian separability. The geometric approach using the convex hull is also a popular method of optimising the SVM algorithm. A study conducted by Mavroforakis ME and Theodoridis S (2006) introduced the concept of reduced convex hull and adapted the Schlesinger-Kozinec algorithm to be applied to both separable and non-separable SVMs. Lastly, clustering-based data reduction approach which removes data points that are unlikely to become a support vector (Suarez et al., 2025). An example study part of this approach is by Shen et al. (2016). Which introduced the usage of parallel hyperplane scheme which combines k-means clustering with recursive hyperplane formation, effectively eliminating redundant points.

### iii. Plans on how to parallelize the proposed process

As mentioned, the proposed project will build upon the implementation presented by Suarez et al. (2025). The project will optimize selected computationally intensive stages of the SVM data-reduction pipeline while retaining the remaining stages of the original workflow. The study by Suarez et al. (2025) identified two main bottlenecks that could exponentially increase time and space complexity, which are the dual variable dimensionality overhead and the sequential point-wise LP overlap overhead. For this project, the group will focus on optimizing the dual variable dimensionality overhead by implementing the following equations using AVX2 SIMD and CUDA instructions:

**Point-in-convex Hull Test Equation**

$$\begin{aligned} \max \quad & \sum_{i=1}^{m} \alpha_i, \\ \text{s.t.} \quad & \sum_{i=1}^{m} \alpha_i x^i = \sum_{j=1}^{n} \beta_j y^j, \\ & \sum_{i=1}^{m} \alpha_i = 1, \quad \sum_{j=1}^{n} \beta_j = 1, \\ & \alpha_i, \beta_j \geq 0. \end{aligned}$$

Figure 1: Point-in-convex Hull Test Equation (image from: Suarez et al. (2025))

NVIDIA CUDA will be used to implement SIMT parallelization for point-in-convex hull tests, allowing independent tests to be executed concurrently on a compatible NVIDIA GPU.

**Barycenter (Geometric) Calculation**

$$z^{B_1} = \frac{1}{m}\sum_{i=1}^{m} x^i,$$

$$z^{B_2} = \frac{1}{n}\sum_{j=1}^{n} y^j.$$

Figure 2: Barycenter (Geometric) Calculation Equation (image from: Suarez et al. (2025))

The barycenter calculation calculates the center point of different classes and identifies if the point in space is within the validity range specified by the Barycenter criteria. x86-64 assembly with AVX2 SIMD instructions and 256-bit YMM registers can be used to parallelize geometric barycenter filtering. These instructions will allow multiple numerical values to be processed within a single vector operation by loading 8 single-precision float values per instruction using VMOVPS, VADDPS, and VDIVSP.

**Barycenter Criteria**

$$\|x - z^{B_j}\|_2 > \|z^{B_i} - z^{B_j}\|_2, \quad i \neq j.$$

Figure 3: Barycenter Criteria Equation (image from: Suarez et al. (2025))

The Barycenter Criteria can be parallelized by performing the euclidean distance operation to find the distance between the two identified barycenters and the distance between point X against a barycenter point. This utilizes the YMM register by loading 8 single precision floats per instruction. Where it performs VMOVPS, VSUBPS, VSQRTPS, and VMULPS for calculating the Euclidean distance. For comparison, the VCMPPS instruction will be utilized to check if the point is within the valid area. Otherwise, the point will be disregarded.

**iv. Other processes in the project that will not be parallelized**

Aside from the processes mentioned above, the linear optimization problem 7 feasibility check accomplished by Suárez et al. (2025) will not be parallelized and will remain sequential.

**c. Outputs of the project**

The final output of the project will be the reduced and filtered training dataset. Additionally, the project will output the performance and execution time speedup metrics of the parallel approach will be compared to the sequential approach programmed in C (

$T = \frac{T_{sequential}}{T_{parallel}}$). Other outputs include the correctness or sanity check of the training dataset filtered through SIMD/SIMT.

**03. Proposed target implementation platform of the proposed project**

Development will be conducted using Visual Studio Code as the primary code editor, GCC or MSVC for compiling C code, NASM for assembling x86-64 assembly routines, CUDA with the nvcc compiler for GPU-based parallelization. The implementation will run on a system with an AVX2-compatible CPU and a CUDA compatible NVIDIA GPU. The final toolchain, operating system, and hardware specifications will be documented once confirmed.

A sequential C implementation will serve as the baseline for performance comparisons, and the reference for validating the correctness of the parallel implementation. All implementations will use equivalent input datasets and workloads. Performance will be evaluated using execution time and speedup, calculated by dividing the sequential C execution time by the parallel implementation's execution time. Correctness checks will account for acceptable floating point differences and vector boundary handling.

**04. Mid-project milestones**

The timeline below serves as our mid-project milestones for the proposed project:

**Milestone #1: Week 7-8:** Baseline implementation of the sequential C program.

- Sequential implementation for all three processes: barycenters (Geometric) calculation, filtering of the barycenters, and point-in-convex hull tests.

**Milestone #2: Week 8-9:** SIMD implementation of barycenters calculation and filtering

**Milestone #3: Week 10:** Mid-project update demo

The continuation serves as the remaining milestones for the project:

**Milestone #4: Week 10-11:** CUDA implementation of point-in-convex hull tests

**Milestone #5: Week 11-12:** Testing of correctness, performance, and speedup metrics

**Milestone #6: Week 13:** Complete project submission and demo

**05. References**

Abdiansah, A., & Wardoyo, R. (2015). Time Complexity Analysis of Support Vector Machines (SVM) in LibSVM. International Journal of Computer Applications, 128(3), 28–34. https://doi.org/10.5120/ijca2015906480

Mavroforakis ME, Theodoridis S (2006) A geometric approach to support vector machine (svm) classification. IEEE Trans Neural Netw 17(3):671–682. https://doi.org/10.1109/TNN.2006.873281

Mun. (2023, February 19). SVMs in Practice: Applications and use cases for machine learning's most effective model. Medium.

https://medium.com/@mun.articles/svms-in-practice-applications-and-use-cases-for-machine-learnings-most-effective-model-2ae25f4207ef

Woodsend, K., & Gondzio, J. (2011). Exploiting separability in large scale linear support vector machine training. Computational Optimization and Applications, 49(2), 241–269. https://doi.org/10.1007/s10589-009-9296-8

Shen X-J, Mu L, Li Z, Wu H-X, Gou J-P, Chen X (2016) Large-scale support vector machine classification with redundant data reduction. Neurocomputing 172:189–197. https://doi.org/10. 1016/j.neucom.2014.10.102

Suárez, C. A., Castro, M., Leon, M., Martin-Barreiro, C., & Liut, M. (2025). Improving SVM performance through data reduction and misclassification analysis with linear programming. Complex & Intelligent Systems, 11(8), 356. https://doi.org/10.1007/s40747-025-01989-4